BST and Hash Table Analysis report

Daniyal Akylbek SE-2530

BST is unbalanced , iterative implementation storing unique integer keys

Hash table separate chainging with fixed capacity 2n+1 load factor a< 0.5 using Math.floorMod

Time complexity

Structure	Operation	Best	Average	Worst
BST	Insert	O(1)	O(log n)	O(n)
BST	Search	O(1)	O(log n)	O(n)
BST	Delete	O(1)	O(log n)	O(n)
Hash table	Insert	O(1)	O(1 + alpha)	O(n)
Hash table	Search	O(1)	O(1 + alpha)	O(n)
Hash table	Delete	O(1)	O(1 + alpha)	O(n)

Results

Insertion order	Elements	Structure	Insert (ms)	Search (ms)	Delete (ms)
random	100	BST	0.009835	0.004597	0.005639
random	100	HashTable	0.003886	0.003576	0.003696
sorted	100	BST	0.009744	0.011737	0.007391
sorted	100	HashTable	0.004657	0.002804	0.002814
random	1000	BST	0.066839	0.050705	0.040430
random	1000	HashTable	0.052328	0.026540	0.008613
sorted	1000	BST	0.634289	1.112206	0.450707
sorted	1000	HashTable	0.020750	0.011017	0.008833
random	10000	BST	0.980341	0.706425	0.770020
random	10000	HashTable	0.277772	0.208739	0.109092
sorted	10000	BST	67.900010	94.505114	58.508888
sorted	10000	HashTable	0.191533	0.057435	0.068802

Key Findings

Random Input Performance: For a random dataset of 10K elements, the Hash Table is roughly 3.4x faster in Search (0.209 ms vs 0.706 ms) and 7x faster in Delete (0.109 ms vs 0.770 ms). For smaller datasets(n = 100), the performance difference between the two structures is minimal. Severe Degradation on Sorted Input: Under sorted input, the unbalanced BST degenerates into a single-linked list of height n-1 (O(n) per operation). At n=10K, BST search time spikes to 94.5051 ms compared to 0.0574 ms for the Hash Table — making the BST search approximately 1,645 times slower.

Separate Chaining Stability: With uniform key distribution and a bounded load factor (a<0.5), bucket chains in the Hash Table remain short, preserving near-constant expected O(1) operational performance regardless of input ordering.

Conclusion Hash Table are optimal for direct key lookups. BSTs are useful when element ordering or range queries are required,but self balancing mechanisms are need to avoid O(n) worst case performance on sorted inputs